Uniform Predecessor Certificates for Intuitionistic Primitive Recursive Arithmetic

A complete small checker and a boundary for initializer fusion

Research prepared for Vladimir Reshetnikov

10 November 2026

Abstract

We give a complete proof presentation of full first-order intuitionistic primitive recursive arithmetic, together with an executable Python checker, both interpretation composites, reproducible whole-program measurements, and a new obstruction to a broad class of proposed arithmetic reductions. The numerical and logical basis is inherited from the supplied drafts. The certificate interface combines constructed predecessor computations with uniform local computation assumptions, so that a submitted counter does not need a separate successor recognizer.

The selected complete checker has 269 normalized tokens, 307 AST nodes, and 629 normalized bytes. Its exact source occupies 373 bytes after gzip compression at level 9. The current finite-corpus minima are 277 tokens, 311 AST nodes, and 672 bytes. These comparisons include every theorem-specific executable component. The main reduction against the unchanged files comes from a simpler exit-status rejection convention. Matched-driver controls are reported explicitly; the source-size result is not a new arithmetic axiomatization or a significant architectural breakthrough.

The new mathematical result extends the existing bounding-coordinate obstruction to arbitrary fixed finite register systems and arbitrarily many fixed probes of one supplied update body. Under an explicit data-flow restriction, every generated term remains bounded by one input coordinate, even with all unary primitive-recursive maps and all bounded primitive-recursive observations freely available. Consequently addition, unrestricted selection, and injective pairing cannot be term-defined in that class. This is a restricted obstruction, not an impossibility theorem for relational or changed-domain bi-interpretations.

Research status. Full iPRA equivalence is retained, and a complete checker smaller than the unchanged current programs is supplied. The requested significant new axiomatic breakthrough has **not** been established. The report distinguishes a measured implementation reduction, inherited equivalence proofs, and the new restricted nonrepresentability theorem.

Contents

1 What the present iteration establishes 3

2 The complete axiomatization 3

3 Why this is full intuitionistic PRA 5

4 The predecessor certificate interface 7

5 Source measurements, gzip, and matched controls 10

6 Validation and reproducibility 11

7 A barrier for finite-state, single-body seed fusion 12

8 What remains open 15

Attribution and complete inherited proofs 16

A Complementary introductions with one local assumption 16

B Arithmetic strength and both ordinary interpretation composites 21

C Internal scalar predecessor bootstrap 23

D From literal-equation induction to full intuitionistic PRA 27

E Pairing, primitive recursion, and internal numerical composites 32

1 What the present iteration establishes

The source comparison is pinned to ProveIt commit

`0300a290bedee5019c974291850b798378438b2e`.

There are 16 current ZIP archives in the supplied directory. Their byte strings were verified against the Git blob identities obtained from the GitHub connector, and the directory was rechecked at this exact revision. The direct and recursive inventories contain respectively 393 and 2,249 Python member occurrences, representing 297 distinct byte strings. Small build scripts, measurement tools, and test drivers are excluded from the complete-checker comparison. Complete checkers included as controls or references are counted. Every source smaller than any claimed minimum was inspected and classified; the identities and measurements are supplied in `evidence/`.

The attainable coordinate minima are 277 significant normalized Python tokens, 311 AST nodes, and 672 normalized UTF-8 bytes. They are attained by different programs. In particular, the 277-token controls already inside the scalar-successor and constructed-computation archives must not be overlooked. A minimum among finitely many submitted implementations is an observed envelope, not a mathematical lower bound on all checkers.

The present artifact makes three separate contributions. First, it implements a direct predecessor-packet interface without the `list(*counter)` successor recognizer used by the flat-computation draft. Constructing the successor makes that recognizer unnecessary. Second, it uses ordinary process exit status for rejection; a successful run also prints `ACCEPT`. An uncaught checking or parsing exception rejects with a nonzero exit status. This removes an otherwise redundant top-level rejection handler. Third, it proves the single-body obstruction of Section 7.

The first contribution is a synthesis of the constructed-computation, transient-seed, and flat-computation interfaces [3, 4, 2]. The initialized scalar iterator, Choice-DI logical rules, endpoint transport, shared continuation, arithmetic bootstrap, and interpretation proofs are inherited. We make no claim to have invented those components. The second contribution is a software-interface simplification, and its effect is removed in the matched-driver comparison. The third extends an existing invariant rather than replacing the established arithmetic basis.

2 The complete axiomatization

2.1 Finite numerical descriptions

There is one numerical sort. Terms are

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only the update h binds u. Other variables in h are passive parameters and stay fixed during the iteration. The two computation schemas, allowed in every retained context, are

$$I(0, a; u.h) = a, \tag{I0}$$
$$I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{IS}$$

These are ordinary scalar iterations, whose update receives the preceding state but not the old counter.

The binder notation has an exact first-order elaboration. Starting with $0, S$, for each fixed earlier-stage term $h(u, \vec{p})$ introduce a symbol $i_h(n, a, \vec{p})$ and the displayed two equations. Expressions and proofs

use only finitely many stages. There is no object variable over function descriptions, and no universally total program evaluator is added. Coherence between alternate finite descriptions follows from the internal uniqueness argument below.

2.2 Logical syntax and inference rules

Native formulas have the grammar

$$A ::= (s = t) \mid Q_y A \mid D_{x,y} A,$$

where the paired binders x, y are distinct. Contexts are finite reusable lists. The designated contradiction is $0 = S0$. The following is the complete primitive rule inventory:

$$\frac{}{\Gamma \vdash A} \quad (A \in \Gamma) \qquad \text{(Hyp)},$$

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \qquad \text{(Tr)},$$

$$\frac{\Gamma \vdash Q_y A(y) \qquad \Gamma \vdash A(0)}{\Gamma \vdash A(t)} \qquad \text{(QE)},$$

$$\frac{\Gamma \vdash D_{x,y} A(x,y) \qquad \Gamma, Q_y A(b,y) \vdash F}{\Gamma \vdash F} \qquad \text{(DE)},$$

$$\frac{\Gamma, A(t,0) \vdash A(t, Sz)}{\Gamma \vdash D_{x,y} A(x,y)} \qquad \text{(DI-case)},$$

$$\frac{\Gamma, E(t,z) \vdash E(t, Sz)}{\Gamma \vdash D_{x,y} E(x,y)} \qquad \text{(DI-progress)}.$$

In Tr, F is any fixed native formula template. All substitutions avoid capture. In DE, b is fresh for the retained context, conclusion, and fixed matrix parameters. In DI, z is fresh for the retained context, witness t, and fixed matrix parameters. Each displayed local assumption is discharged. The progress matrix E must be a *literal equation*; an equation underneath a binder is not an equation matrix for this rule.

I0 and IS are axiom schemas. There are no additional primitive equality, successor, induction, or ordinary logical axioms. In particular, compound progress is not an admissible extra rule: it would supply induction for arbitrary formulas, exceeding the intended induction restriction.

The equivalent certificate presentation uses a computation theorem as a local assumption:

$$\frac{\Gamma, H \vdash A}{\Gamma \vdash A} \quad \text{(Supply)}, \qquad H \text{ an instance of I0 or IS}.$$

This is justified by ordinary-assumption cut. Conversely, Supply followed by Hyp proves a computation axiom. The checker does not import a library of theorem schemas beyond the two explicitly implemented computations.

2.3 Structural admissibility and the zero-predecessor case

Weakening, exchange, contraction, renaming, numerical substitution, and ordinary-assumption cut are admissible by induction on finite derivations. Before insertion or substitution, rename local eigenparameters away from the inserted terms and retained data. Numerical substitution preserves

the root constructor of an induction matrix, so it preserves the equation-only restriction. At a cut hypothesis leaf insert the supplied derivation; every other rule commutes with the construction.

Let $B(p) := I(0, p; v.0)$. I0 gives $B(p) = p$. Tr, using this equation as both premises with template $w = p$, gives $p = p$. The templates $w = s$, $s = w$, and $f(s) = f(w)$ then give symmetry, transitivity, and numerical congruence. These are derivations, not extra equality rules.

Lemma 2.1 (Boxed zero suffices)**.** *I0 and successor computations whose predecessor has a nonempty recursor representation derive every instance of IS.*

Proof. For any term p, compute at the composite predecessor $B(p)$:

$$I(SB(p), a; u.h) = h[I(B(p), a; u.h)/u].$$

Transport along $B(p) = p$ with the fixed template $I(Sw, a; u.h) = h[I(w, a; u.h)/u]$. This yields the desired IS instance. The argument applies to every domain element and uses no standardness assumption. In the implementation, the only canonical predecessor that needs this repair is literal zero; all nonempty canonical predecessors already select the successor branch. □

3 Why this is full intuitionistic PRA

All metatheorems in this section are inherited and are reproduced in full in the appendices, with attribution [8, 2]. They concern derivability on the whole numerical domain, with actual equality.

3.1 Deriving induction, cases, and ordinary logic

Use an unused existential coordinate in DI-progress, then DE and QE. This derives contextual equation induction:

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}.$$

The same construction with DI-case derives zero/successor cases for arbitrary native formulas from proofs of their zero and successor instances. Contexts may contain arbitrary formulas; only an induction matrix is restricted to an equation.

Define $C(v, a, b) = I(v, a; u.b)$. Its endpoint equations are $C(0, a, b) = a$ and $C(Sv, a, b) = b$. By external recursion on finite formula syntax define a gate

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(D_{x,y}A) := D_{x,y}G_vA.$$

An auxiliary Q can first be represented by D with an unused existential coordinate. Then G_0A is provable and $G_{Sv}A$ is interderivable with A. Padding A by $D_{u,y}G_yA$ preserves it.

For equation leaves set $M_v(s = t, r = w) := (C(v, s, r) = C(v, t, w))$. For paired formulas align their binders and recurse; if only one input has a binder, pad the equation input first. The maximum remaining binder depth decreases. Binder congruence, proved with DI-case, establishes $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $1 = S0$, $2 = S1$, $\delta(v) = C(v, 1, 0)$, and $p(z) = I(z, 0; u.\delta(u))$. Only $p(1) = 1$ and $p(2) = 0$ are

needed. With fresh unused coordinates, the full logical interface is

$$\begin{aligned} A \to_* B &:= D_{u,z} M_z(A, B), \\ A \wedge_* B &:= D_{u,z} G_z M_{p(z)}(A, B), \\ \exists_* x\, A(x) &:= D_{x,y} G_y A(x), \\ A \vee_* B &:= \exists_* x\, M_x(A, B), \\ \forall_* x\, A(x) &:= A(0) \wedge_* D_{u,z} G_z A(z), \\ \bot_* &:= (0 = 1). \end{aligned}$$

The appendix proves every introduction and elimination rule. For example, implication introduction uses the retained zero selector as its assumption; elimination specializes the resulting closure at 1. Conjunction projections specialize at $1, 2$. Existential elimination uses a fresh witness, the zero-gate theorem, and specialization at 1.

Numerical cases yield $(x = 0) \vee_* \exists_* y(x = Sy)$. Its equality-based elimination is essential when the target occurs in a retained context: one adds an equality and transports, instead of silently substituting into the retained context. Under $0 = 1$, transport through C yields any numerical equality, and external formula induction with DI-case gives explosion for every native formula. No excluded middle for arbitrary quantified formulas is used.

3.2 Arithmetic strength and the ordinary realization

Let T_I use ordinary intuitionistic first-order equality logic with genuine falsity, the staged iterator symbols, I0, IS, equation induction, and the explicit axioms

$$x = 0 \vee \exists y(x = Sy), \qquad (0 = S0) \to \bot.$$

The falsity bridge must be present: positive numerical equations alone have a singleton ordinary model. Interpreting native falsity as $0 = S0$ does not justify omitting that bridge from the ordinary comparison theory.

The inherited arithmetic construction follows this noncircular order: predecessor from scalar iteration and equation induction; a characteristic of actual equality on the entire domain; characteristics of fixed quantifier-free formulas and hence quantifier-free induction; then pairing and a compiler for arbitrary fixed primitive-recursive descriptions. The complete finite predecessor terms and their identities are in Appendix C. Pairing is not assumed in order to construct predecessor.

For a target recursion with base g and step h, using the proved pairing π and projections L, H, the compiler takes

$$V(n, \vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

An equation induction proves $L(V(n, \vec{p})) = n$; the two projection identities give the required base and step equations. Passive parameters remain fixed. The compiler is an external recursion on finite descriptions, not an internal universal evaluator.

Theorem 3.1 (Whole-domain bi-interpretability)**.** *The native calculus is a faithful presentation of an ordinary theory bi-interpretable and equiconsistent with full first-order iPRA. Both numerical domain composites are internally isomorphic to the identity by the relation $x = y$.*

Proof. Interpret numerical iteration in iPRA by primitive recursion ignoring the counter in the update. Interpret formulas by

$$(Q_y A)^\circ = A^\circ(0) \to \forall y A^\circ(y), \quad (D_{x,y} A)^\circ = \exists x\big(A^\circ(x, 0) \to \forall y A^\circ(x, y)\big),$$

and preserve equality. Case DI is valid by coverage and progress DI by equation induction. All other native rules preserve these readings. In the reverse direction, the displayed logical macros, equality reflection, quantifier-free induction, and scalar compiler interpret every ordinary symbol and axiom of iPRA.

Two iterations with the same seed and provably equal fixed updates are equal at every counter by equation induction. The same uniqueness argument applies to general primitive recursion. External induction on a finite description therefore identifies its double translation with the original operation. This also proves coherence of the staged binder notation under substitution. Domains are the whole original domains, and equality is actual equality, so the graph $x = y$ is internally total, bijective, and preserves all interpreted operations.

For formulas, the nontrivial native return translation is

$$D_{x,y}A \dashv\vdash \exists_* x\big(A(x,0) \to_* \forall_* y A(x,y)\big).$$

Forward, eliminate D at a fresh witness and use its closure under the temporarily assumed base. Reverse, eliminate the translated existential and use its implication and universal instance at Sz to discharge DI-case. The Q round trip is its unused-witness instance. External induction on formula syntax gives both full formula composites. The explicit falsity bridge preserves contradictions; effective finite proof translations give equiconsistency, without claiming an internal proof of consistency. □

4 The predecessor certificate interface

The numerical decoder and logical checking operations are inherited from the flat-computation checker. The changed arithmetic instruction is

$$[\texttt{null}, p, a, h].$$

It constructs the following local equation from its three data fields:

$$\begin{cases} I(0,a;u.h) = a, & p \text{ has a false raw representation,} \\ I(Sp,a;u.h) = h[I(p,a;u.h)/u], & p \text{ has a true raw representation.} \end{cases}$$

The condition is on the finite representation, not on whether the value of the predecessor is zero in a model. A nonempty representation may denote zero. The source is always explicitly zero or a constructed successor, so a separate check that a supplied counter has successor shape is unnecessary.

Canonical literal zero uses the base branch. To obtain the successor step at zero, use the nonempty predecessor $B(0)$ and Lemma 2.1. Every other canonical predecessor, including an allocated rigid parameter whose value may be zero, selects a valid successor computation directly. This proves canonical completeness of the arithmetic interface independently of Python truth values.

4.1 The shared raw representation

The public input is one JSON array. Its last element is a requested closed formula. Earlier four-field instructions are consumed right to left; context membership closes a goal without reading an instruction. A major proof is checked before its continuation. An unread prefix is inert.

The same finite tree representation is read in two sorts. Numerically, an empty list is zero, a unary list is successor, a binary list is zero with checked annotations, and a ternary list is an iterator. As a formula, these are respectively truth $0 = 0$, Q, equality, and D (whose first two fields are checked

annotations). A ternary numerical node gives its counter and seed two dummy zero slots and its update two aliases of its one state slot. This does not add a counter argument to the update.

Opening carries a depth d, even at formula positions and odd at numerical positions. Unary, binary, and ternary nodes add respectively $2, 1, 4$. At numerical depth $2q + 1$, coordinates $2j - 1, 2j$ name the same jth protected slot; coordinate zero denotes zero. Negative coordinates name allocated rigid numerical parameters. A global floor starts at zero and decreases once per continuation. A scalar coordinate is retained only when it lies between this floor and the current depth. Otherwise a numerical position uses the supplied image, itself checked at numerical root, while formula padding completes to zero. Missing used numerical images reject. The full input/alias contract is in `code/FORMAT.txt`.

The raw codec is compact but less legible than the mathematical syntax. The separately executable readable checker documents the same operations. It is included in the measurements and is not passed off as the compact program. Neither implementation imports a theorem library, compiler, or arithmetic evaluator.

4.2 Soundness of uniform computation supply

The only new proof obligation relative to the inherited logical kernel is the arithmetic source constructor. It is useful to make the treatment of unused state slots explicit.

Lemma 4.1 (Dummy zero slots)**.** *For any fixed successfully traversed raw numerical expression, inserting dummy zero slots outside its local binders does not change its zero-completed numerical meaning. Admitted rigid negative parameters may have arbitrary values.*

Proof. Induct on the finite raw tree. An exposed coordinate is completed to zero in both readings; an internal protected coordinate keeps its local slot meaning; a rigid parameter keeps its value. Unary numerical nodes add the same dummy zero, and binary numerical nodes are zero in both readings. For a ternary node the induction hypothesis identifies the counter and initializer and identifies the updates at every state. Iterator uniqueness then identifies the iterations. This is an argument for each fixed finite expression, not an assumed uniformly total evaluation function. □

Theorem 4.2 (Uniform predecessor supply)**.** *Every successfully supplied computation equation is a theorem of the native calculus, uniformly in every admitted new parameter. No known-zero-witness discharge is required.*

Proof. For a null major, the common continuation opens the generated equation with numerical image zero. All newly admitted rigid parameters remain rigid under this opening.

If the raw predecessor is false, successful traversal makes its numerical meaning zero. Both occurrences of the initializer agree by the dummy-slot lemma, and the generated equation is I0.

If the predecessor is true, the constructed source counter is its unary wrapper, hence a genuine successor regardless of the predecessor's numerical value. The proposed recursive image is the iterator at that predecessor, the same initializer, and the same update. If opening the update uses that image, the image is first checked at the old parameter floor. The inherited opening/substitution lemma identifies the resulting right side with the update at that preceding iterator. Inserting the extra dummy slot for the source successor does not alter the predecessor's zero-completed meaning. The equation is IS.

If opening never uses the recursive image, the update is independent of that exposed state coordinate. Any meaningful use of an as-yet unallocated parameter in the update would enter the same substitution branch, so such a use is also absent. The predecessor and initializer may acquire a fresh parameter, but

IS with a state-independent update is valid for every predecessor and initializer. Thus the equation is again an ordinary computation theorem uniformly in that parameter.

For canonical sources the argument is immediate substitution in I0/IS; the case distinction above additionally covers successful noncanonical zero-completed representations. Unlike the older packet, no arbitrary non-successor term can be mistaken for a successor counter: the wrapper is constructed by the checker itself. □

Theorem 4.3 (Checker correspondence)**.** *Every successful execution proves its decoded closed native goal. Every closed native theorem has a finite canonical certificate accepted when the checking computation has sufficient resources.*

Proof. Induct on a finite successful execution. Membership is Hyp. A checked equality or Q major with matching endpoints implements Tr or QE. If an optional endpoint request does not match, it leaves the old goal to be proved and adds only truth. A checked D major opens its Q clause at a fresh rigid parameter. The two introduction choices are exactly DI-case and literal-equation DI-progress. The decreasing floor validates the retained goal and witness before allocating a new parameter. Uniform computation supply is Theorem 4.2. A malformed scalar formula has no finite closing rule that creates a new substantive formula: context entries are the traversed formula lists generated by these cases. These facts give a finite native derivation by ordinary cut.

For completeness, rename every local binder and eigenparameter in a native derivation. Use the inherited named encoding of numerical and formula syntax, emit its logical rule instructions, and replace each computation axiom by Supply followed by Hyp. A literal-zero predecessor in IS is replaced by the boxed proof of Lemma 2.1. All other canonical computation sources are direct. Reverse the instruction stream and append the closed goal. Ordinary structural induction on the derivation establishes acceptance; the supplied producer exercises this construction on explicit finite examples and is not a universally verified compiler. □

4.3 A measurable benefit in proof certificates

The simpler source constructor has a concrete benefit even though its matched kernel is one byte larger. In the old flat interface, a scalar parameter predecessor was not list-shaped, so the delivered encoder used three command records: $B(p) = p$, computation at $B(p)$, and transport. The new interface checks computation at that parameter directly in one record. The arbitrary parameter need not denote a positive number; the constructed *source counter* is Sp in either case.

The same named derivations, emitted by the frozen old and new producers and accepted by their respective complete checkers, give:

Named derivation	Old records	New records
Nonvacuous zero witness	4	2
Atomic progress closure	6	4
Arbitrary-term specialization	8	6
Contextual induction with seed congruence	48	40

These are reproducible counts for the supplied encoders, not lower bounds on the shortest proofs in either calculus. The 48-to-40 example saves about 16.7% of the command records. The literal-zero predecessor still uses three records in both delivered encodings. The complete 14-example comparison includes serialized byte counts as well as command counts.

Physical Python stack and memory limits can reject very large valid certificates. This is a resource limitation of the implementation, not an extra object-theory axiom. Canonical well-founded proofs do not depend on divergent malformed-input aliases. The selected checker retains explicit list dispatch rather than using recursion-limit-dependent string decoding.

5 Source measurements, gzip, and matched controls

All measurements count the whole standalone module: the JSON import, opening and validation, numerical computations, logical inference, parameter discipline, input driver, and success output. Python and its standard JSON library are shared runtime infrastructure. No theorem-specific executable dependency is omitted. Docstrings and comments are removed, `ast.unparse` is applied, and a final newline is added. Significant tokens, every AST node, and UTF-8 bytes are then counted with CPython 3.12.14, as in the supplied corpus.

Complete program	Tokens	AST	Bytes	Gzip-9
Current token leader	277	317	684	403
Current AST leader	280	311	705	393
Current byte leader	281	322	672	397
Current flat computation	278	314	679	402
Current predecessor joint	280	314	684	390
New predecessor, two verdicts	278	315	680	399
Selected complete checker	269	307	629	373
Readable complete checker	312	359	1531	1024
Ancillary byte profile	269	311	616	366
Ancillary AST profile	268	300	649	367

The same rejection convention applied to the old programs gives:

Complete program	Tokens	AST	Bytes	Gzip-9
Current token leader (matched driver)	268	309	633	379
Current AST leader (matched driver)	271	303	654	374
Current byte leader (matched driver)	272	314	621	371
Current flat computation (matched driver)	269	306	628	374
Current predecessor joint (matched driver)	271	306	637	366

The Bytes column is normalized-source size; Gzip-9 is the size of the exact raw source, including any comments in the readable version. The ancillary profiles combine previously published Python representation choices with the new packet. They are included for comparison, not selected as the clearest implementation. The AST profile accepts extra empty-object aliases and rejects nonempty strings through Python's finite recursion limit; it could diverge on such strings in an ideal unbounded interpreter. The primary checker has explicit list dispatch and avoids that particular limitation.

Gzip is an informational metric, not an optimization objective. The exact procedure is

```
gzip.compress(source_bytes, compresslevel=9, mtime=0)
```

The full gzip member, including header and trailer, is counted. Exact raw and normalized-source gzip sizes and hashes are in `evidence/checker_metrics.json`. The primary compact source is already normalized, so those two sizes coincide. The delivered `.py.gz` decompresses exactly to its measured Python source. The runtime and zlib version are recorded for reproducibility.

The exit-status convention is part of the comparison, not hidden in it. Removing `except: print('REJECT'); raise` preserves the accepted inputs: every exception still exits nonzero, and every success still prints `ACCEPT`. The same modification is applied to each frozen leading comparator in the matched table. Therefore the below-corpus whole-file result does not imply a smaller mathematical basis. The predecessor constructor also replaces an unusual eager unpacking check with direct source construction; its exact source-cost difference is shown rather than described as a large gain.

5.1 The entire selected executable

For inspection, this is the whole measured file. There are no omitted theorem-specific imports or generated axiom tables.

```
from json import load
k = 0
i = lambda a, t=None, d=0: [i(b, t, d + b' \x02\x01\x04'[len(a)]) for b in a] if isinstance(a, list)
    else a if k <= a < d else d % 2 and i(t, d=1)

def c(a, g):
    global k
    if i(a) in g:
        return
    m, s, u, t = q()
    if m:
        c(m, g)
        if m in ([s, u], not s and [t]) and i(t, u) == a:
            a = i(t, s)
        h = m[2:]
    elif m is None:
        h = [[s and [s], u, t], i(t, [s, u, t], 1) if s else u]
    else:
        h = a = i(a[2], t, 2)
    k -= 1
    c(i(a, [k]), g + [i(h, k if m or len(a) % 2 == m else 0)])
q = load(open(0)).pop
c(q(), [])
print('ACCEPT')
```

6 Validation and reproducibility

All 248 complete-program expectations passed (31 cases, four implementations, ordinary and optimized Python). The suite also passed 24 malformed-input runs and three targeted mutation controls. Both old and new encodings were executed in 28 certificate-comparison runs. The independent arithmetic diagnostic attempted 27,600 packets, generated 15,168 equations, and checked 60,672 parameter valuations; it found zero counterexamples and recorded 0 evaluation-budget skips.

The proof examples exercise computation with nonzero seeds, actual state use, passive parameters, equality transport under binders, Q and D rules, both introduction choices, and zero-predecessor repair. Negative cases exercise false arithmetic conclusions, scope leaks, missing images, malformed input, and an attempted compound-progress inference. Boundary cases whose intended arithmetic meaning changed with the new predecessor packet are reclassified by that meaning; an old malformed counter is not automatically a malformed predecessor.

The numerical falsification tests evaluate the generated equations under an independent standard-model decoder. They can find a bad transcription or counterexample but cannot establish a whole-domain interpretation. The mathematical conclusions rest on the written proofs, including the attributed appendices. There is no Lean/Rocq formalization or external human peer review in this package. Independent assistant reviews are preserved as reviews, not represented as machine-checked metatheorems.

To reproduce the delivered evidence from a fresh extraction:

```
python3 reproduce.py
python3 code/checker.py < tests/fixtures/I0_arbitrary_seed.json
python3 build.py
```

The first command verifies file identities, recomputes the whole-program and gzip metrics, and runs the delivered finite checks without changing the payload. The second checks a single certificate. The third rebuilds the modular and standalone LaTeX report with an installed PDFLaTeX distribution and no shell escape. These operations need no network.

7 A barrier for finite-state, single-body seed fusion

The current tied-seed obstruction [4, 5] concerns the particular scalar recurrence $B_h(0) = 1 + h(0,0)$ and $B_h(m+1) = h(m, B_h(m))$. The following extension shows that the obstacle is not removed by finitely many initialization probes, finitely many internal registers, counter-dependent probe locations, or arbitrary fixed primitive-recursive processing of the probe results. The essential restriction is that all passive data and all retained state reach the seed and update combiners only through evaluations of *one fixed body*. This is a statement about numerical terms in the standard model; it is not an impossibility theorem for relational or changed-domain interpretations.

7.1 The exact class of constructors

For a function $f : \mathbb{N}^r \to \mathbb{N}$, call i a bounding coordinate if some nondecreasing unary primitive-recursive function H satisfies

$$f(x_1, \dots, x_r) \le H(x_i) \quad \text{for every } (x_1, \dots, x_r). \tag{BC}$$

A globally bounded function is admitted separately, including at arity zero.

Start with variables and zero. One may additionally admit *every* unary primitive-recursive operation and *every* globally bounded primitive-recursive operation of arbitrary finite arity. More generally, any primitive operation already satisfying (BC) may be admitted. Ordinary finite term composition is allowed.

A single-body constructor is specified by fixed finite dimensions and fixed primitive-recursive maps. Its only supplied subdescriptions are a counter term t and one body $h(\vec{u}, \vec{p})$, where $\vec{u}$ is a fixed finite tuple of local variables and $\vec{p}$ lists the passive variables. Write $n = t(\vec{p})$. For fixed integers $a, b, d, q \ge 1$, take

$$A_i : \mathbb{N} \longrightarrow \mathbb{N}^q \qquad (1 \le i \le a),$$
$$U_j : \mathbb{N}^{d+2} \longrightarrow \mathbb{N}^q \qquad (1 \le j \le b),$$
$$F_0 : \mathbb{N}^a \longrightarrow \mathbb{N}^d, \qquad F_1 : \mathbb{N}^b \longrightarrow \mathbb{N}^d, \qquad G : \mathbb{N}^d \longrightarrow \mathbb{N}.$$

The constructor denotes the value $G(\vec{r}_n)$ determined by

$$\vec{r}_0 = F_0\big(h(A_1(n), \vec{p}), \ldots, h(A_a(n), \vec{p})\big), \tag{1}$$

$$\vec{r}_{m+1} = F_1\big(h(U_1(n, m, \vec{r}_m), \vec{p}), \ldots, h(U_b(n, m, \vec{r}_m), \vec{p})\big). \tag{2}$$

The maps may differ between constructor schemes, but each is fixed before its body is supplied. They are not additional object-language input functions. No passive parameter enters F_0, F_1, G directly: all such information must pass through an evaluation of the same body h. The target counter n and elapsed counter m may enter the probe locations A_i, U_j, exactly as displayed. The readout receives the final registers, not an additional independent data argument.

Every supplied body is earlier finite syntax; nesting constructors is allowed. Thus each fixed description has a primitive-recursive standard interpretation, by structural induction followed by primitive recursion on m. No uniform evaluator for all primitive-recursive descriptions is postulated.

The previously studied tied-seed constructor is the special case $d = 1$, $q = 2$, $a = b = 1$, $A_1(n) = (0, 0)$, $U_1(n, m, r) = (m, r)$, $F_0(z) = z + 1$, $F_1(z) = z$, and $G(r) = r$. The present class also allows many probes at distinct locations, register vectors, and seed probes whose locations depend arbitrarily on the target counter.

Theorem 7.1 (Single-body information barrier)**.** *Every finite term generated by the preceding primitives, composition, and single-body constructors is globally bounded or has a bounding coordinate in the sense of* (BC)*. Given bounding-coordinate or global-bound witnesses for the primitive operations in its finite description, the proof effectively constructs a suitable coordinate and a nondecreasing primitive-recursive bound. The existence statement does not require these witnesses to be supplied as additional language data.*

Proof. Induct on finite descriptions, treating a body's local and passive variables as independent inputs when applying the induction hypothesis. The primitive cases hold by assumption. Composition preserves the property: if the outer function is bounded in its jth input, compose its nondecreasing bound with the bound for the jth actual argument. A globally bounded actual argument gives a constant bound.

It remains to analyze (1)–(2). For any fixed primitive-recursive map with finitely many input and output coordinates, a nondecreasing unary primitive-recursive envelope bounds all its output coordinates whenever all its input coordinates are at most the envelope's argument. Such an envelope is obtained by taking a maximum over a finite box. Fix envelopes f_0, f_1, g for F_0, F_1, G, respectively.

If h is globally bounded by C, every probe value is at most C. Every register, at every stage, is therefore at most $\max(f_0(C), f_1(C))$. Applying g gives a constant bound for the result, independently of the counter and passive parameters.

Suppose next that a passive variable p_i bounds h:

$$h(\vec{u}, \vec{p}) \le H(p_i) \quad \text{uniformly in all } \vec{u}, \vec{p}.$$

All seed and update probes obey the same bound, even though their local arguments depend on the counters and registers. Hence the final result is at most

$$g\big(\max(f_0(H(p_i)), f_1(H(p_i)))\big).$$

This is a nondecreasing primitive-recursive bound depending on the one passive coordinate p_i alone.

The remaining case is that one local coordinate u_ℓ bounds h:

$$h(\vec{u}, \vec{p}) \le H(u_\ell) \quad \text{uniformly in the passive parameters.}$$

Choose nondecreasing primitive-recursive envelopes a and u such that all coordinates of every $A_i(n)$ are at most $a(n)$, and all coordinates of every $U_j(n, m, \vec{r})$ are at most $u(s)$ whenever $n, m, r_1, \ldots, r_d \le s$. Define by primitive recursion

$$\begin{aligned} v(0, n) &= f_0(H(a(n))), \\ v(m+1, n) &= \max\left(v(m, n), f_1\big(H(u(n + m + v(m, n)))\big)\right). \end{aligned}$$

Induction on m proves that every coordinate of $\vec{r}_m$ is at most $v(m, n)$. At the base, all local probe arguments are bounded by $a(n)$. At the step, the nonnegative sum $n + m + v(m, n)$ bounds the two counters and every register. Apply the bounds for U_j, then H, then F_1. Consequently the result is at most $g(v(n, n))$. Its monotone envelope

$$K(n) = \max_{z \le n} g(v(z, z))$$

is unary primitive recursive and independent of every passive parameter.

Apply the induction hypothesis to the actual counter term $t(\vec{p})$. If it is bounded by a constant c, the whole constructor is bounded by $K(c)$. If $t(\vec{p}) \le N(p_i)$ for a bounding coordinate p_i, then its result is at most $K(N(p_i))$. This is the required one-coordinate bound. The cases exhaust the possible induction hypotheses for h, completing the proof. All envelopes, finite maxima, compositions, and recursions in the argument are effective operations on fixed primitive-recursive descriptions, relative to the supplied primitive bound witnesses. A bound for an arbitrary globally bounded PR function cannot in general be computed from its bare program description. Unary PR primitives have effective finite-box envelopes, and Boolean predicates have known bound one, so those extensions need no additional bound oracle. □

Corollary 7.2 (What these repairs cannot recover)**.** *No finite term in this class represents ordinary addition, an unrestricted numerical two-branch selector, or an injective map $\mathbb{N}^2 \longrightarrow \mathbb{N}$. This remains true even if all unary primitive-recursive functions and all bounded primitive-recursive predicates are freely available as primitives.*

Proof. Fixing either argument of addition at zero leaves its output unbounded in the other argument, so addition has no bounding coordinate. For a selector $C(0, a, b) = a$, $C(Sn, a, b) = b$, fixing the control at zero leaves a unbounded; fixing $a = 0$ and the control at one leaves b unbounded; fixing $b = 0$ and the control at zero leaves a unbounded. Thus none of its three coordinates bounds it. Finally, for a binary injection, fixing a supposed bounding coordinate at zero would map an infinite fiber injectively into a finite bounded subset of $\mathbb{N}$, a contradiction. A global bound fails for the same reasons. □

7.2 The exact boundary and the useful design lesson

The theorem is not a prohibition on recursion with finitely many registers. Its crucial hypothesis is the data interface in (1)–(2). For example, if a passive initializer a is permitted to enter a register directly, the one-register loop

$$r_0 = a, \qquad r_{m+1} = Sr_m, \qquad \text{output } r_n$$

already computes $a + n$, which has no bounding coordinate. Even while keeping initialization tied to a body, direct register retention escapes the theorem: take $r_0 = h(0, \vec{p})$ and $r_{m+1} = Sr_m$. For the constant body $h(u, a) = a$, the output is again $a + n$. This update is deliberately outside (2), because

the old register reaches the update combiner without first passing through the body. Such changes may be good designs, but they genuinely change the data interface rather than merely probe the same body more elaborately.

The theorem therefore rules out a substantial family of natural repairs to the tied-seed proposal: adding a fixed number of probe locations, finite internal registers, arbitrary unary decoders, arbitrary bounded observations, or more elaborate fixed processing of the probe outputs cannot by itself yield a term compiler for iPRA's primitive-recursive signature. It does not rule out a second independent body, an independent data input, direct register retention, graph-defined functions, a quotient or changed domain, or a different logical interpretation. In particular, no conclusion about all relational bi-interpretations follows from this standard-model term obstruction.

8 What remains open

The retained numerical basis already has only zero, successor, and one scalar-iteration constructor. The logical presentation has equality and one paired binder, with Q a definitional convenience. Its apparent brevity rests on substantial derivations of predecessor, equality reflection, ordinary logic, and the primitive-recursion compiler. Counting just a list of primitive rule names would hide that explanatory burden.

The newer successor-seeded scalar basis removes the separate successor constructor, but its predecessor and logical bootstrap have already been completed in the current drafts [6, 7, 4]. Repeating those proofs would not be a new iteration. The tied-seed variants have a genuine data-flow barrier; the new theorem extends its scope but expressly leaves other interpretations open.

Deleting atomic DI-case is also not a valid simplification of the unrestricted existing formula language. Changing the public language to gated atomic matrices can evade that particular obstruction, but it introduces new grammar or decoder obligations and has no demonstrated complete-checker advantage here. Similarly, a graph-defined selector does not by itself extend an old recursion schema to terms using that new function. Such a schema extension needs its own conservativity proof.

An absolute minimum, a significantly smaller mathematical architecture, and a substantial improvement under a fully matched checker convention remain open. The appropriate conclusion is the concrete implementation and restricted theorem proved above, not a claim that the user's broader research objective has been solved.

Attribution and complete inherited proofs

The following five sections are reproduced unchanged from the flat-computation archive at the pinned revision. That archive traces them to the generated-assumptions, named-specialization, and earlier scalar developments. Their exact source hashes are in `evidence/inherited_provenance.json`. The upstream MIT No Attribution license is included at `reference/PROVEIT_LICENSE.txt`; mathematical attribution is retained throughout. The sections supply the complete written bridge used by Theorem 3.1.

A Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

A.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)}\ \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0,1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

A.2 Exact equivalence and the unused-witness construction

Lemma A.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem A.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y} E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition A.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \le 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

A.3 Induction and numerical cases separate cleanly

Lemma A.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma A.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

A.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A, B),\\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A, B),\\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_* x\, M_x(A, B),\\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z),\\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

A.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x (B^\circ(x, 0) \to \forall y B^\circ(x, y)),$$

$$\langle A; y.B \rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y} B \dashv\vdash \exists_* x (B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a,0)$ that clause supplies $B(a,y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* y B$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem A.2 preserves every native sequent.

A.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

B Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x,0) \to \forall y B^\circ(x,y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section A recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

B.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem B.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

C Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [9] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

C.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0,1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n+2) = N(n). \tag{3}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n+1) = 1, \qquad N(3n) = N(n). \tag{4}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

C.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{5}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n) + 1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n+2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{6}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{7}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (7) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{8}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (7); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

C.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{9}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (8), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{10}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{11}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \le y$ is motivation, not a premise of the following argument.

C.4 Saturated iteration gives predecessor

We now need only the three equations (11). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (12) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (13) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (14)$$

The base is (13). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (15)$$

The first uses the comparator recurrence. The second uses (14) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (16)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \qquad (17)$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (16) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

C.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

D From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [10]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix C, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

D.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{18}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{19}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x \, (x = 0 \vee x = S(Px)), \tag{20}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{21}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p} \, \left[(E(0, \vec{p}) \wedge \forall n (E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n \, E(n, \vec{p}) \right]. \tag{22}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (22) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0,\vec{p}), \qquad \Gamma, E(z,\vec{p}) \vdash E(Sz,\vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n,\vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

D.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u,0,1), \qquad K(u) = C(u,1,Pu). \tag{23}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{24}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{25}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (18). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (24); symmetry and (21) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{26}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{27}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{28}$$

To prove the range assertion, split (20). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (19) also give, for all u, v,

$$C(u,v,v) = v. \tag{29}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{30}$$

If $u = 1 = S0$, (25) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

D.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{31}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{32}$$

Using these equations and (22), we now derive

$$0 + y = y, \tag{33}$$
$$(Sx) + y = S(x + y), \tag{34}$$
$$d(Sx, Sy) = d(x, y), \tag{35}$$
$$d(Sa, a) = 1. \tag{36}$$

Each induction below has a single equation as its induction formula.

For (33), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (34), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (34) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{37}$$

Indeed, both sides are $S(y + r)$.

For (35), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (36). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (35) and the induction hypothesis.

D.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C(\mathrm{sg}(d(x,y)),\ x,\ y + d(x,y)) = x. \tag{38}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (33) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (29).

For the step, assume (38) at y and abbreviate

$$r = d(x,y), \qquad r' = Pr = d(x,Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (38) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (37) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (38). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa,b) = 1 \quad \longleftrightarrow \quad a = b. \tag{39}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (38). If $d(Sa,b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (36) give $d(Sa,b) = d(Sa,a) = 1$.

Define the equality characteristic by the fixed description

$$e(a,b) = \mathrm{sg}\big(K(d(Sa,b))\big). \tag{40}$$

The sign, countdown, and residual reflection lemmas give

$$e(a,b) = 0 \quad \longleftrightarrow \quad a = b, \tag{41}$$

$$e(a,b) = 0 \ \vee\ e(a,b) = 1. \tag{42}$$

In detail, $e(a,b) = 0$ is equivalent to $K(d(Sa,b)) = 0$ by (27), which is equivalent to $d(Sa,b) = 1$ by (30), and then to $a = b$ by (39). The range assertion is (26) applied to $K(d(Sa,b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a,b)=0$ branch, (41) gives $a=b$. In the $e(a,b)=1$ branch, an assumption $a=b$ would give $e(a,b)=0$ and therefore $0=1$, contradicting separation. Hence

$$\forall a\,\forall b\,(a=b\vee(a=b\to\bot)) \tag{43}$$

is a theorem, not an extra hypothesis.

D.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t}=e(s,t), \qquad b_\bot=1, \tag{44}$$
$$b_{A\wedge D}=C(b_A,b_D,1), \tag{45}$$
$$b_{A\vee D}=C(b_A,0,b_D), \tag{46}$$
$$b_{A\to D}=C(b_A,b_D,0). \tag{47}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$(b_A=0\vee b_A=1)\quad\text{and}\quad(b_A=0\longleftrightarrow A). \tag{48}$$

Here are the full connective checks. In every case $b_A=1$ implies $\neg A$: if A held, the already established implication $A\to b_A=0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (41)–(42). For falsity the range assertion is $1=0\vee 1=1$, with its right disjunct proved by reflexivity. Also $1=0\to\bot$ follows from separation and symmetry, while $\bot\to 1=0$ follows by explosion. Thus $1=0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A=0\vee b_A=1$. In the zero branch, A holds and $b_{A\wedge D}=b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A\wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D}=1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A\wedge D$; conversely an assumption $A\wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D}=0$.

Disjunction. In the branch $b_A=0$, the formula A holds and $b_{A\vee D}=0$. The range is immediate, and both $b_{A\vee D}=0$ and $A\vee D$ hold, proving their equivalence. In the branch $b_A=1$, we have $\neg A$ and $b_{A\vee D}=b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A\vee D$. Conversely, from $A\vee D$, the A alternative contradicts $\neg A$ and yields $b_D=0$ by explosion, while the D alternative yields $b_D=0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (48) for every quantifier-free formula.

D.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (48), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

E Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [11]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

E.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free

inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n)+1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a, b) = (a+b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a, b)) = a + b$, and therefore

$$L(\pi(a, b)) = a, \qquad H(\pi(a, b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

E.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

E.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section A give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

References

[1] Vladimir Reshetnikov, repository maintainer. *ProveIt: current draft collection*, commit `0300a290bedee5019c974291850b798378438b2e`. https://github.com/VladimirReshetnikov/ProveIt/tree/0300a290bedee5019c974291850b798378438b2e/docs/drafts. All archive references below refer to this immutable collection.

[2] *Flat computation for intuitionistic primitive recursive arithmetic.* `ipra_flat_computation_2026-11-10.zip`, in [1]. Source of the null-tag uniform computation interface and the complete attributed logical/arithmetic appendices used here.

[3] *Constructing computation sources for iPRA.* `ipra_constructed_computation_2026-11-10.zip`, in [1].

[4] *A one-state recursor for iPRA.* `ipra_transient_seed_2026-11-10.zip`, in [1]. Source of the predecessor-packet joint profile, the 311-node comparator, and a tied-seed bounding-coordinate obstruction.

[5] *Counter congruence suffices for iPRA.* `ipra_counter_congruence_2026-11-10.zip`, in [1]. Source of the 672-byte complete control and related initializer obstructions.

[6] *Scalar successor recursion for intuitionistic PRA.* `ipra_scalar_successor_2026-11-10.zip`, in [1].

[7] *Completing the successor-seeded scalar basis for iPRA.* `ipra_successor_seed_completion_2026-11-10.zip`, in [1].

[8] *One assumption at a time: a smaller iPRA proof kernel*, Choice-DI development, reproduced with attribution in the named-specialization, generated-assumptions, and flat-computation archives of [1].

[9] Inherited scalar predecessor development, reproduced as `reference/scalar_predecessor.tex` in [2] and in this report.

[10] Inherited forward-left equality-reflection development, reproduced as `reference/equality_reflection.tex` in [2] and in this report.

[11] Inherited progress-closure pairing and primitive-recursion compiler, reproduced as `reference/primitive_recursion.tex` in [2] and in this report.